Kesimpulan

- Random Forest menghasilkan error yang lebih kecil dibanding Linear Regression.
- Analisis dapat dikembangkan, misalnya dengan memperluas kota yang diuji atau mengubah tugas menjadi klasifikasi kondisi cuaca.

Uji Coba Model

```
# import numpy as np
```

```
uji_coba = ([[80, 1013, 5, 25, 12, 15, 1, 2026]]) # Added hour, day, month, year
uji_coba
```

```
[[80, 1013, 5, 25, 12, 15, 1, 2026]]
```

```
import pandas as pd

# Get feature names from the training data X
feature_columns = X.columns.tolist()

# Convert uji_coba to a DataFrame with explicit column names
uji_coba_df = pd.DataFrame(uji_coba, columns=feature_columns)

nilai_uji_coba_lr = lr.predict(uji_coba_df)
nilai_uji_coba_lr
```

```
array([280.84674447])
```

9. J.62DMI00.014.1 – Mengevaluasi Hasil Pemodelan

Tahap terakhir yakni mengevaluasi hasil pemodelan menggunakan MAE (Mean Absolute Error) dan RMSE (Root Mean Squared Error)

```
results
```

	Model	MAE	RMSE
0	Linear Regression	4.514873	5.776936
1	Random Forest	0.885598	1.330608

9. J.62DMI00.014.1 – Mengevaluasi Hasil Pemodelan

Tahap terakhir yakni mengevaluasi hasil pemodelan menggunakan MAE (Mean Absolute Error) dan RMSE (Root Mean Squared Error)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# Prediksi
lr_pred = lr.predict(X_test)
rf_pred = rf.predict(X_test)

# Evaluasi
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest'],
    'MAE': [
        mean_absolute_error(y_test, lr_pred),
        mean_absolute_error(y_test, rf_pred)
    ],
    'RMSE': [
        np.sqrt(mean_squared_error(y_test, lr_pred)),
        np.sqrt(mean_squared_error(y_test, rf_pred))
    ]
})
```

8. J.62DMI00.013.1 – Membangun Model

Pada tahap ini, digunakan dua model:

1. Linear Regression
2. Random Forest Regressor

```
# Model 1: Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```
# Model 2: Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

8. J.62DMI00.013.1 – Membangun Model

Pada tahap ini, digunakan dua model:

1. Linear Regression
2. Random Forest Regressor

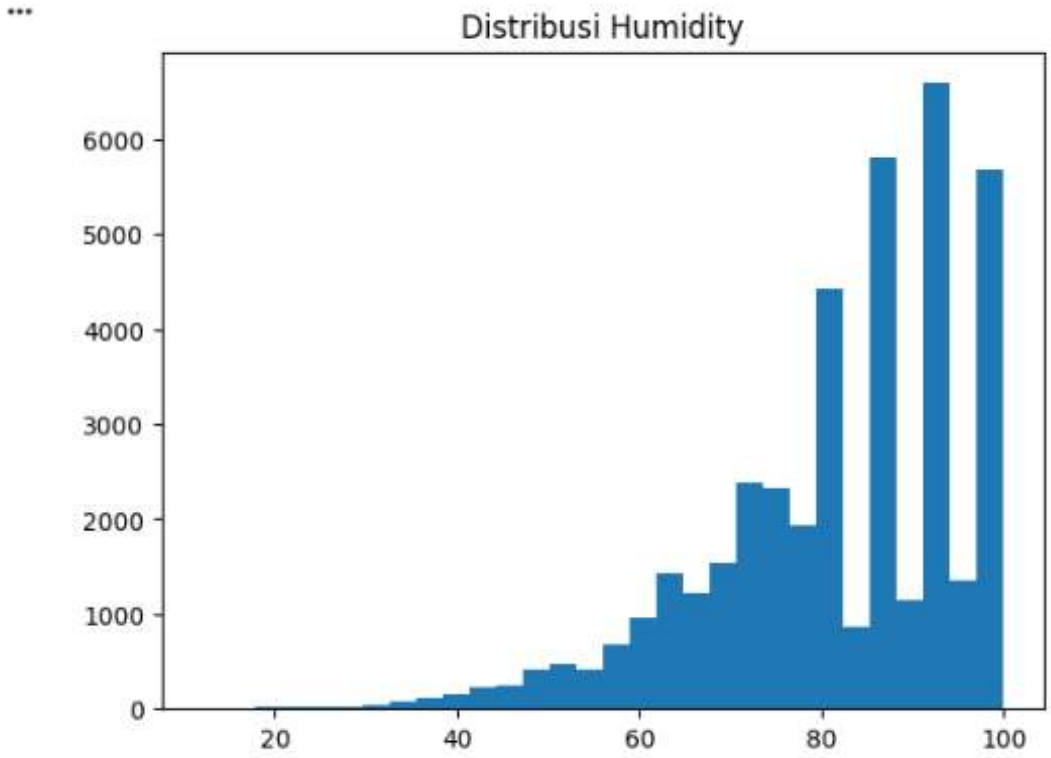
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

7. J.62DMI00.010.1 – Menentukan Label Data

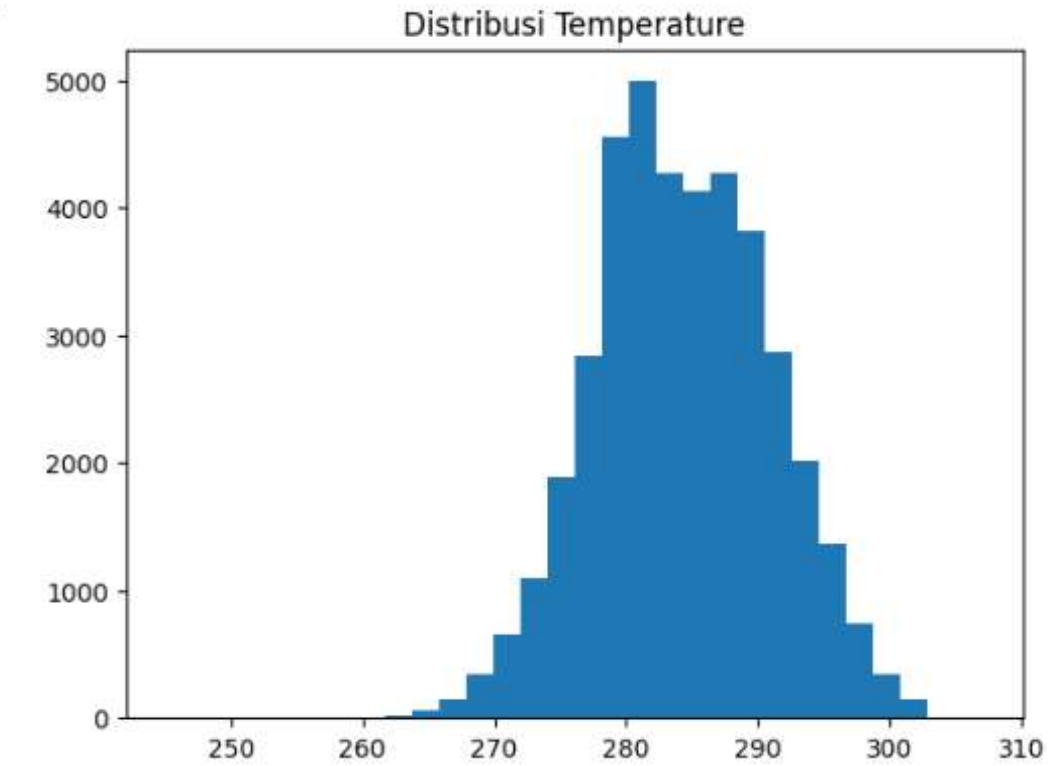
Label (target) adalah **temperature**. Fitur adalah variabel cuaca lainnya. Didefinisikan y sebagai **temperature**, x sebagai variabel yang lain.

```
X = df[['humidity', 'pressure', 'wind_speed', 'wind_dir', 'hour', 'day', 'month', 'year']]
y = df['temperature']
```

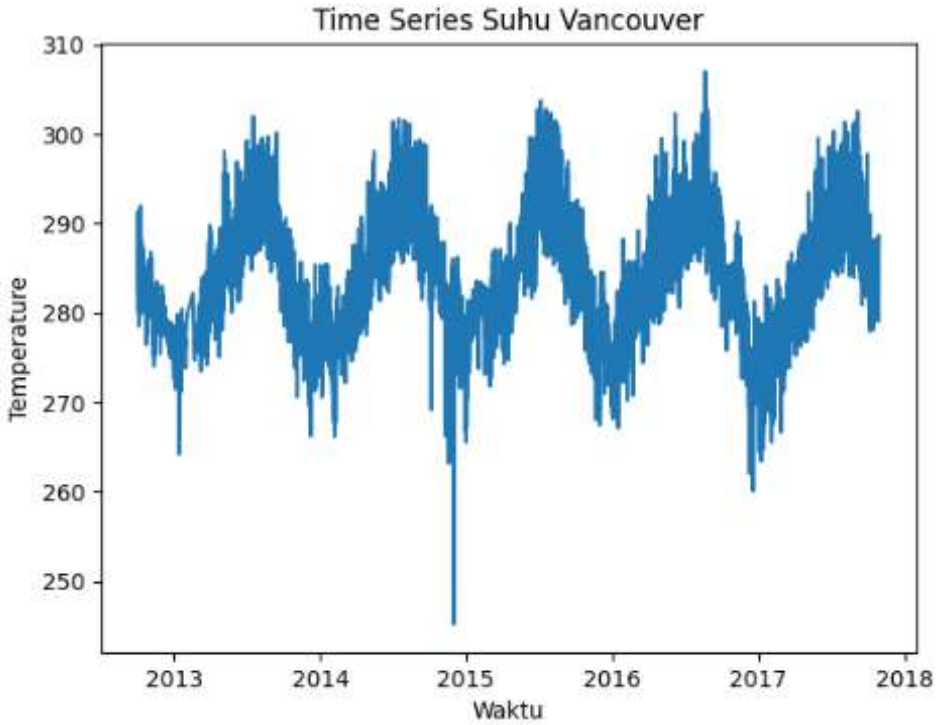
```
plt.figure()
plt.hist(df['humidity'], bins=30)
plt.title('Distribusi Humidity')
plt.show()
```



```python
plt.figure()
plt.hist(df['temperature'], bins=30)
plt.title('Distribusi Temperature')
plt.show()
```



```python
import matplotlib.pyplot as plt

plt.figure()
plt.plot(df['datetime'], df['temperature'])
plt.title('Time Series Suhu Vancouver')
plt.xlabel('Waktu')
plt.ylabel('Temperature')
plt.show()
```



6. J.62DMI00.009.1 – Mengkonstruksi Data

Menambahkan fitur berbasis waktu (jam, hari, bulan, tahun). Kemudian membuat grafiknya.

```
df['hour'] = df['datetime'].dt.hour
df['day'] = df['datetime'].dt.day
df['month'] = df['datetime'].dt.month
df['year'] = df['datetime'].dt.year

df
```

	datetime	temperature	humidity	pressure	wind_speed	wind_dir	weather	hour	day	month	year
21	2012-10-02 09:00:00	284.590217	87.0	807.0	0.0	268.0	broken clouds	9	2	10	2012
22	2012-10-02 10:00:00	284.588174	88.0	849.0	0.0	281.0	broken clouds	10	2	10	2012
23	2012-10-02 11:00:00	284.586130	89.0	890.0	0.0	295.0	broken clouds	11	2	10	2012
24	2012-10-02 12:00:00	284.584087	89.0	932.0	0.0	309.0	broken clouds	12	2	10	2012
25	2012-10-02 13:00:00	284.582043	90.0	973.0	0.0	323.0	broken clouds	13	2	10	2012
...	...	...	...	...	...	...	...	...	...	...	...
44456	2017-10-27 20:00:00	286.440000	93.0	1024.0	2.0	280.0	few clouds	20	27	10	2017
44457	2017-10-27 21:00:00	287.870000	93.0	1023.0	2.0	340.0	sky is clear	21	27	10	2017
44458	2017-10-27 22:00:00	288.420000	82.0	1022.0	4.0	300.0	few clouds	22	27	10	2017
44459	2017-10-27 23:00:00	288.650000	87.0	1022.0	0.0	272.0	sky is clear	23	27	10	2017
44460	2017-10-28 00:00:00	288.010000	87.0	1021.0	4.0	290.0	few clouds	0	28	10	2017

40624 rows × 11 columns

5. J.62DMI00.008.1 – Membersihkan Data

Masih menggunakan tabel dataframe kota Vancouver, pada tahap ini dilakukan

- Menghapus missing values
- Mengurutkan waktu

```
df = df.dropna()
df = df.sort_values('datetime')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 40624 entries, 21 to 44460
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   datetime     40624 non-null  datetime64[ns]
 1   temperature  40624 non-null  float64
 2   humidity     40624 non-null  float64
 3   pressure     40624 non-null  float64
 4   wind_speed   40624 non-null  float64
 5   wind_dir     40624 non-null  float64
 6   weather      40624 non-null  object
dtypes: datetime64[ns](1), float64(5), object(1)
memory usage: 2.5+ MB
```

4. J.62DMI00.007.1 – Menentukan Objek Data

Pada tahap ini, objek data ditentukan sebagai **data cuaca time-series per kota**.
Untuk pemodelan, dipilih satu kota contoh: **Vancouver**.

```
city_name = 'Vancouver'

df = pd.DataFrame({
    'datetime': pd.to_datetime(temperature['datetime']),
    'temperature': temperature[city_name],
    'humidity': humidity[city_name],
    'pressure': pressure[city_name],
    'wind_speed': wind_speed[city_name],
    'wind_dir': wind_dir[city_name],
    'weather': weather[city_name]
})

df
```

	datetime	temperature	humidity	pressure	wind_speed	wind_dir	weather
0	2012-10-01 12:00:00	NaN	NaN	NaN	NaN	NaN	NaN
1	2012-10-01 13:00:00	284.630000	76.0	NaN	0.0	0.0	mist
2	2012-10-01 14:00:00	284.629041	76.0	NaN	0.0	6.0	broken clouds
3	2012-10-01 15:00:00	284.626998	76.0	NaN	0.0	20.0	broken clouds
4	2012-10-01 16:00:00	284.624955	77.0	NaN	0.0	34.0	broken clouds
...	...	...	...	...	...	...	...
45248	2017-11-29 20:00:00	NaN	NaN	NaN	NaN	NaN	NaN
45249	2017-11-29 21:00:00	NaN	NaN	NaN	NaN	NaN	NaN
45250	2017-11-29 22:00:00	NaN	NaN	NaN	NaN	NaN	NaN
45251	2017-11-29 23:00:00	NaN	NaN	NaN	NaN	NaN	NaN
45252	2017-11-30 00:00:00	NaN	NaN	NaN	NaN	NaN	NaN

45253 rows × 7 columns

3. J.62DMI00.006.1 – Memvalidasi Data

Pada tahap ini, akan di cek Missing values (.isna) dan Duplikasi data (.duplicated) ke semua dataset

```
city.isna().sum()
```

	0
City	0
Country	0
Latitude	0
Longitude	0

dtype: int64

```
city.duplicated().sum()
```

```
np.int64(0)
```

2. J.62DMI00.005.1 – Menelaah Data

Pada tahap ini, dilakukan telaah struktur data, tipe data, dan contoh isi data menggunakan **.info** dan **.head** ke semua dataset.
.info untuk menampilkan informasi umum dari tabel
.head untuk menampilkan 5 baris teratas dari tabel

```python
city.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   City       36 non-null     object
 1   Country    36 non-null     object
 2   Latitude   36 non-null     float64
 3   Longitude  36 non-null     float64
dtypes: float64(2), object(2)
memory usage: 1.3+ KB
```

```python
city.head()
```

	City	Country	Latitude	Longitude
0	Vancouver	Canada	49.249660	-123.119339
1	Portland	United States	45.523449	-122.676208
2	San Francisco	United States	37.774929	-122.419418
3	Seattle	United States	47.606209	-122.332069
4	Los Angeles	United States	34.052231	-118.243683

1. J.62DMI00.004.1 – Mengumpulkan Data

Pada tahap ini, seluruh dataset CSV dimasukkan ke dalam Python menggunakan pandas.

[45]
✓ 0s

```python
# Import library pandas

import pandas as pd
```

[46]
✓ 1s

```python
# Memuat 7 datasets

city = pd.read_csv('city_attributes.csv')
humidity = pd.read_csv('humidity.csv')
pressure = pd.read_csv('pressure.csv')
temperature = pd.read_csv('temperature.csv')
weather = pd.read_csv('weather_description.csv')
wind_dir = pd.read_csv('wind_direction.csv')
wind_speed = pd.read_csv('wind_speed.csv')
```